Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Guía de utilización de la herramienta SIWA TOOLCAHIN GUI

Autor:

Iván Loaiza Arce

Fecha:

10 de marzo de 2026

Proyecto: Microcontrolador SIWA — TEC-RISC-V

Tema: Manual de uso para validación, compilación y generación de archivos

Introducción

El presente documento tiene como objetivo describir el uso de la herramienta SIWA TOOLCAHIN GUI, desarrollada para facilitar el análisis, validación, compilación y generación de archivos asociados al flujo de trabajo del microcontrolador SIWA basado en la arquitectura RISC-V RV32I.

La herramienta fue implementada en Python con interfaz gráfica, de modo que el usuario pueda seleccionar un archivo fuente en lenguaje C, verificar su compatibilidad con las restricciones de la arquitectura y generar archivos intermedios y finales como `.s`, `.elf`, `.hex` y `.bin`, sin necesidad de ejecutar manualmente todos los comandos desde la terminal.

Esta guía describe los requisitos de uso, la función de cada sección de la interfaz y el procedimiento general para trabajar con la aplicación.

Objetivo de la herramienta

La herramienta SIWA TOOLCAHIN GUI fue diseñada para asistir al usuario en el proceso de desarrollo de software para la arquitectura SIWA. Su objetivo principal es automatizar tareas del toolchain y reducir errores durante la validación y compilación de programas en C destinados al microcontrolador.

Entre sus funciones principales se encuentran:

- Seleccionar archivos fuente en lenguaje C.
- Analizar el código según restricciones de la arquitectura SIWA.
- Generar archivos ensamblador `.s`.
- Generar ejecutables `.elf`.
- Convertir ejecutables a formatos `.hex` y `.bin`.
- Mostrar mensajes de error, advertencia y salida de compilación en una consola integrada.

Requisitos previos

Para utilizar correctamente la herramienta es necesario contar con los siguientes elementos instalados en el sistema:

- Python 3.x

- Toolchain de RISC-V accesible desde el PATH del sistema
- Compilador cruzado, por ejemplo `riscv64-unknown-elf-gcc`
- Utilidad `objcopy`
- Opcionalmente, `qemu-system-riscv32` para simulación

Además, el usuario debe disponer de un archivo fuente en lenguaje C y, si desea un control más preciso del mapa de memoria, de un archivo `linker.ld`.

Dependencias a utilizar

Para utilizar correctamente la herramienta SIWA TOOLCAHIN GUI es necesario contar con ciertos programas instalados en el sistema. En el desarrollo del proyecto se utilizó el sistema operativo Ubuntu, por lo que a continuación se describe el proceso de instalación de cada uno de los requisitos.

Instalación de Python 3

La herramienta fue desarrollada en Python, por lo que es necesario tener instalado Python 3 en el sistema.

En Ubuntu, Python generalmente ya se encuentra instalado por defecto. Para verificar la instalación se puede utilizar el siguiente comando en la terminal:

```
python3 --version
```

Si Python no está instalado, se puede instalar utilizando el gestor de paquetes del sistema:

```
sudo apt update
sudo apt install python3
```

También se recomienda instalar el gestor de paquetes `pip` para Python:

```
sudo apt install python3-pip
```

Instalación del toolchain de RISC-V

Para compilar programas destinados a la arquitectura RISC-V es necesario instalar un compilador cruzado. En este proyecto se utiliza el toolchain basado en `riscv64-unknown-elf`.

Una forma sencilla de instalar el toolchain en Ubuntu es mediante los paquetes disponibles en el repositorio:

```
sudo apt install gcc-riscv64-unknown-elf
```

Este paquete incluye herramientas fundamentales como:

- `riscv64-unknown-elf-gcc`
- `riscv64-unknown-elf-objcopy`
- `riscv64-unknown-elf-objdump`

Para verificar que el compilador esté disponible en el sistema se puede utilizar:

```
riscv64-unknown-elf-gcc --version
```

Verificación del PATH del sistema

La herramienta GUI invoca directamente las herramientas del toolchain, por lo que estas deben encontrarse disponibles en el `PATH` del sistema.

Para comprobar si el compilador está accesible desde cualquier directorio se puede utilizar:

```
which riscv64-unknown-elf-gcc
```

Si el comando devuelve una ruta, significa que el compilador se encuentra correctamente configurado en el sistema.

Instalación de QEMU para simulación (opcional para codigos que no lleven arquitectura de SIWA)

Para ejecutar programas compilados en un entorno de simulación se puede utilizar QEMU. En particular, la herramienta utiliza el ejecutable `qemu-system-riscv32` para simular un sistema RISC-V.

La instalación en Ubuntu se puede realizar con el siguiente comando:

```
sudo apt install qemu-system-misc
```

Luego se puede verificar la instalación ejecutando:

```
qemu-system-riscv32 --version
```

Si el comando devuelve la versión instalada, significa que el entorno de simulación está disponible para ser utilizado por la herramienta.

Descarga del repositorio del proyecto

El código fuente de la herramienta SIWA TOOLCHAIN GUI se encuentra disponible en el repositorio del proyecto en GitHub. Para utilizar la herramienta es necesario descargar este repositorio en el sistema.

La forma recomendada de hacerlo es utilizando el sistema de control de versiones `git`.

Instalación de Git

Si el sistema no tiene instalado `git`, se puede instalar utilizando el gestor de paquetes de Ubuntu con el siguiente comando:

```
sudo apt install git
```

Para verificar que la instalación se realizó correctamente se puede ejecutar:

```
git --version
```

Clonar el repositorio

Una vez instalado Git, el repositorio del proyecto puede descargarse utilizando el comando `git clone`. Este comando crea una copia local del repositorio en el sistema.

```
git clone https://github.com/I-L-A/Proyecto-Compilador-SIWA.git
```

Después de ejecutar el comando, se creará un directorio llamado `Proyecto-Compilador-SIWA` que contiene todos los archivos del proyecto.

```
 ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain  git clone https://github.com/I-L-A/Proyecto-Compilador-SIWA.git
Cloning into 'Proyecto-Compilador-SIWA'...
remote: Enumerating objects: 52, done.
remote: Counting objects: 100% (52/52), done.
remote: Compressing objects: 100% (42/42), done.
remote: Total 52 (delta 19), reused 37 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (52/52), 2.53 MiB | 5.38 MiB/s, done.
Resolving deltas: 100% (19/19), done.
 ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain  ls
Proyecto-Compilador-SIWA
 ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain  cd Proyecto-Compilador-SIWA
 ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain/Proyecto-Compilador-SIWA  main  LS
zsh: command not found: LS
 ✗ ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain/Proyecto-Compilador-SIWA  main  ls
 Comandos   Compilador   Documentacion  'Ejemplos C'   GUI   README.md
 ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain/Proyecto-Compilador-SIWA  main  cd GUI
 ivan@DESKTOP-8O1OQMN  ~/TFG2026/siwa_toolchain/Proyecto-Compilador-SIWA/GUI  main  python3 SIWA_TOOLCHAIN_GUI.py
```

Figura 1: Clonado del repositorio del proyecto SIWA y acceso al directorio desde la terminal.

Acceder al directorio del proyecto

Una vez descargado el repositorio, se puede acceder al directorio del proyecto utilizando el comando:

```
cd Proyecto-Compilador-SIWA
```

Dentro de este directorio se encuentran los archivos necesarios para ejecutar la herramienta, incluyendo el script principal de la interfaz gráfica desarrollado en Python.

Estructura del repositorio

El repositorio contiene diferentes carpetas utilizadas durante el desarrollo del proyecto, entre ellas:

- `Compilador` — contiene los archivos relacionados con la herramienta de compilación.
- `Comandos` — incluye ejemplos de comandos utilizados en el toolchain.
- `Ejemplos C` — contiene programas en lenguaje C utilizados para pruebas del compilador.
- `GUI` — contiene la interfaz gráfica del sistema desde donde se ejecuta la herramienta SIWA TOOLCHAIN.

Una vez descargado el repositorio y ubicados dentro del directorio del proyecto, y tras haber instalado previamente Python, el toolchain de RISC-V y opcionalmente QEMU, el sistema estará listo para ejecutar la herramienta gráfica SIWA TOOLCHAIN GUI. A partir de este punto, el usuario podrá iniciar el proceso de compilación de programas para la arquitectura SIWA utilizando las funciones de validación, compilación y generación de archivos que proporciona la herramienta.

Descripción general de la interfaz

La interfaz de SIWA TOOLCAHIN GUI está organizada en diferentes campos y botones que permiten configurar el proceso de validación y compilación.

- Campo de selección del archivo `.c`: permite cargar el programa fuente.

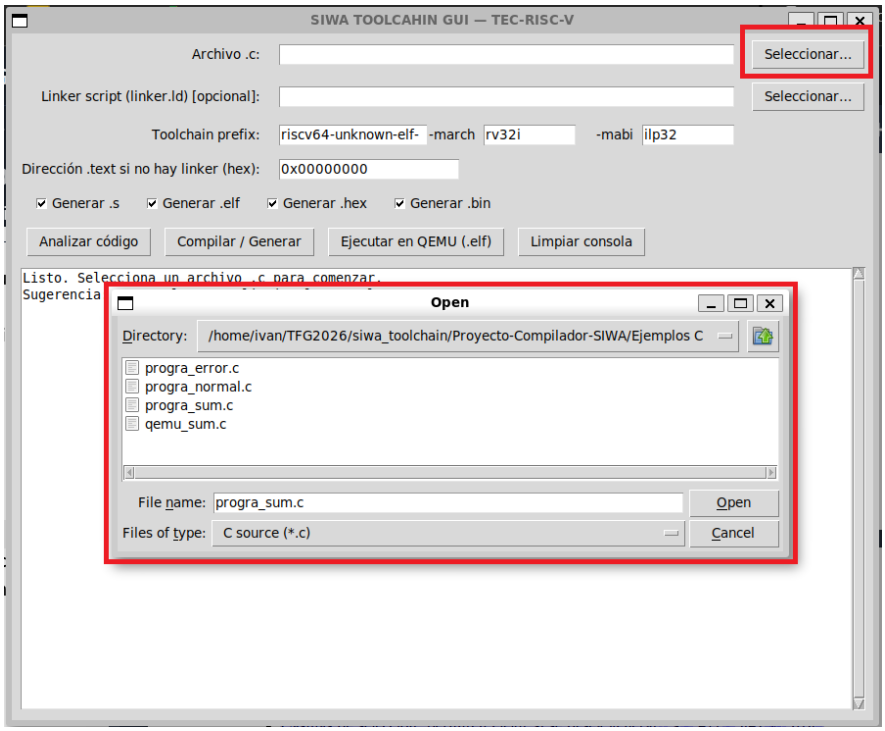

Figura 2: Selección de archivo .c en el GUI.

- Campo de selección del `linker.ld`: permite indicar el script de enlace.

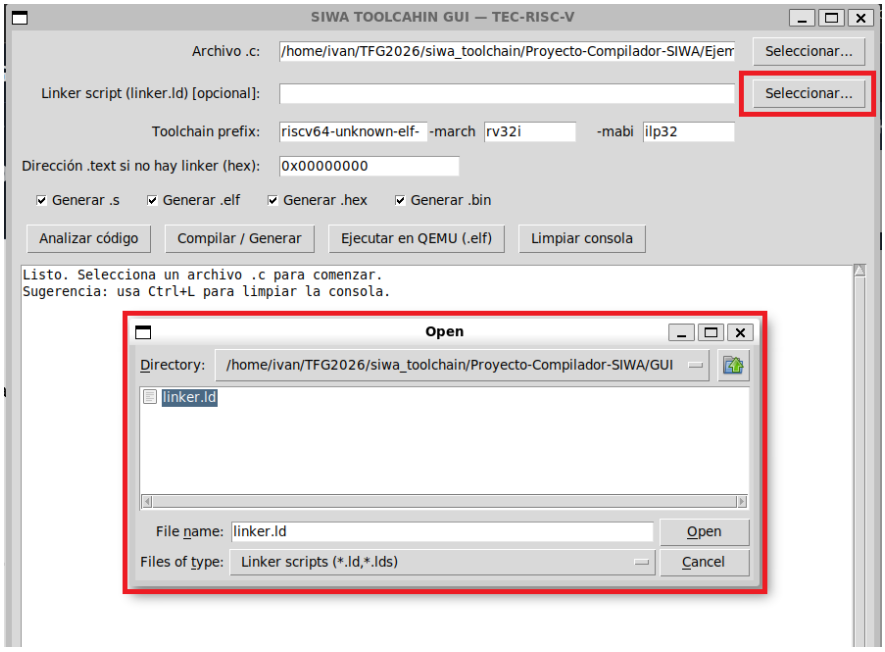

Figura 3: Selección de archivo linker en el GUI.

- Campo del prefijo del toolchain: define el compilador y utilidades a utilizar.

- Campos `-march` y `-mabi`: permiten definir la arquitectura y ABI objetivo.
- Campo de dirección `.text`: se utiliza cuando no se proporciona un linker script.

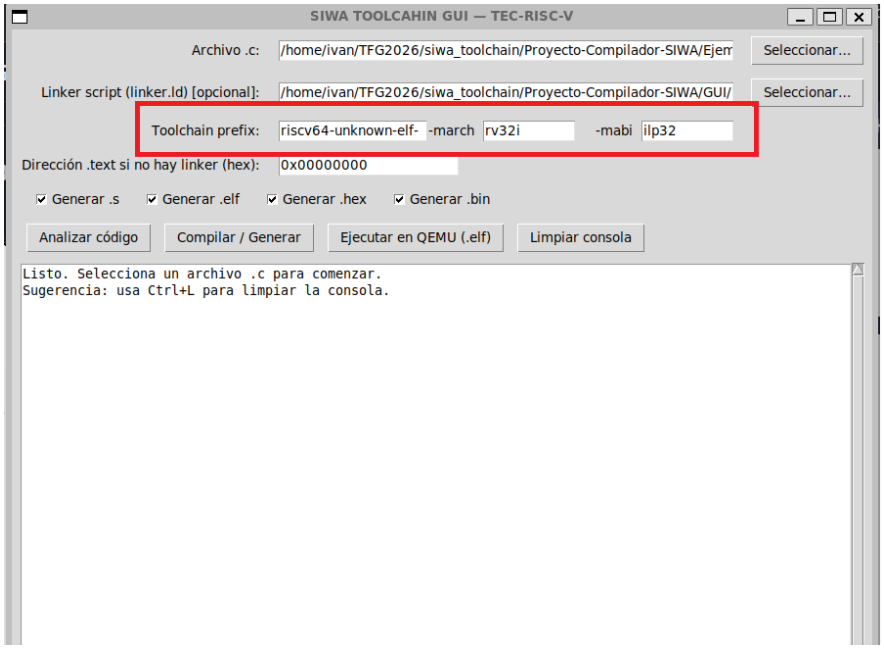

Figura 4: Definicio de utilidades en el GUI.

- Casillas de selección: permiten elegir si se desea generar `.s`, `.elf`, `.hex` y `.bin`.

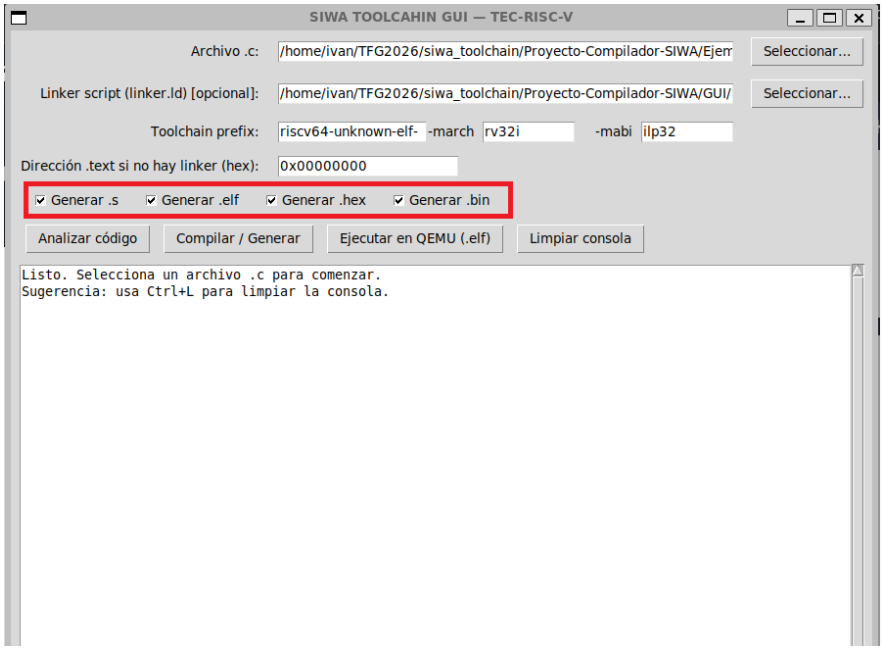

Figura 5: Elección de archivos a crear en el GUI.

- Botón de análisis: revisa el código y reporta restricciones o errores.

Figura 6: Selección de analizar código en el GUI.

Para esta parte es muy importante destacar que es donde el GUI analiza el código .C y revisa sus limitaciones, seguidamente un ejemplo con dos códigos .C uno con errores y otro correcto para el SIWA, donde se ve reflejada la funcionalidad de analizar código.

Para el código con errores tenemos los siguientes códigos en .C:

Listing 1: Ejemplo de programa con múltiples errores para la arquitectura SIWA

```
#include <stdio.h>
#include <stdlib.h>

#define GPIO_BASE 0x00008000  // Direcci n fuera del rango
   permitido
#define GPIO (*(unsigned int*)GPIO_BASE)  // Puntero sin volatile

int main() {
    int a = 25;
    int b = 10;
    int resultado = a + b;

    printf("El resultado es: %d\n", resultado);  // Funci n
       prohibida
```

```
    GPIO = resultado;

    __asm__("fence");  // Instrucci n no soportada

    unsigned int val;
    __asm__("csrr␣%0,␣0x300" : "=r"(val));  // CSR fuera del
       rango permitido

    return 0;  // El programa no debe retornar (no hay sistema
       operativo)
}
```

Listing 2: Ejemplo de programa en C utilizado para pruebas del toolchain

```
#include <stdio.h>

int main() {
    int a = 12;
    int b = 5;
    int resultado;

    // Realiza la suma
    resultado = a + b;

    // Muestra el resultado por consola
    printf("El␣resultado␣de␣la␣suma␣es:␣%d\n", resultado);

    return 0;
}
```

Resultado con el primer código

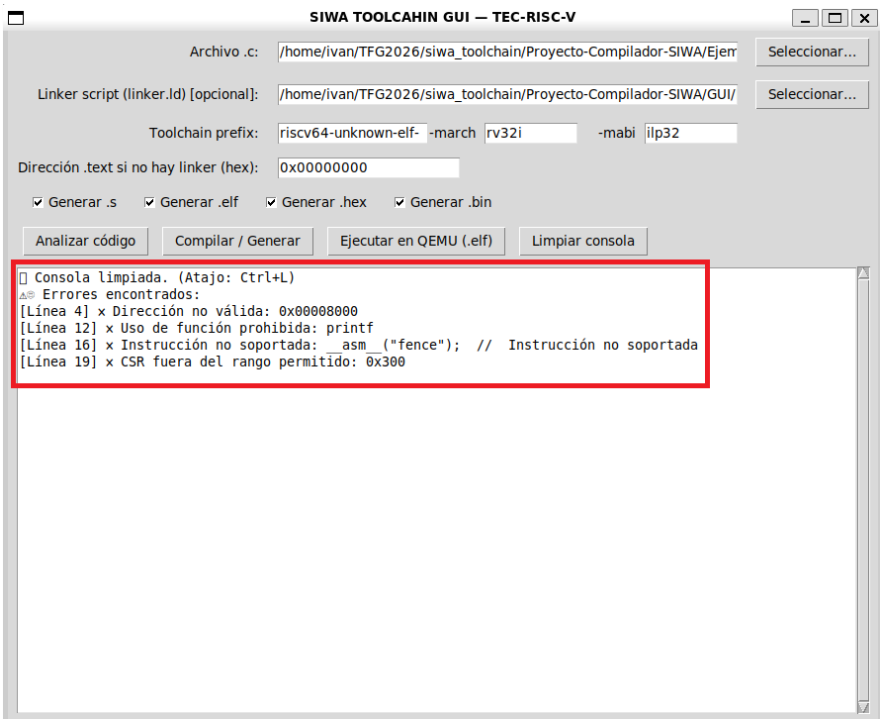

Figura 7: Resultado del código programa con múltiples errores para la arquitectura SIWA en el GUI.

En el ejemplo mostrado en la Figura 7, la herramienta detecta varios errores, entre ellos el uso de una dirección de memoria no válida (`0x00008000`), el uso de la función prohibida `printf`, la presencia de una instrucción ensamblador no soportada (`fence`) y el acceso a un registro CSR fuera del rango permitido (`0x300`).

Este proceso permite identificar problemas en el código antes de iniciar la compilación, evitando errores posteriores en el toolchain y facilitando la corrección del programa por parte del usuario.

Resultado con el segundo código

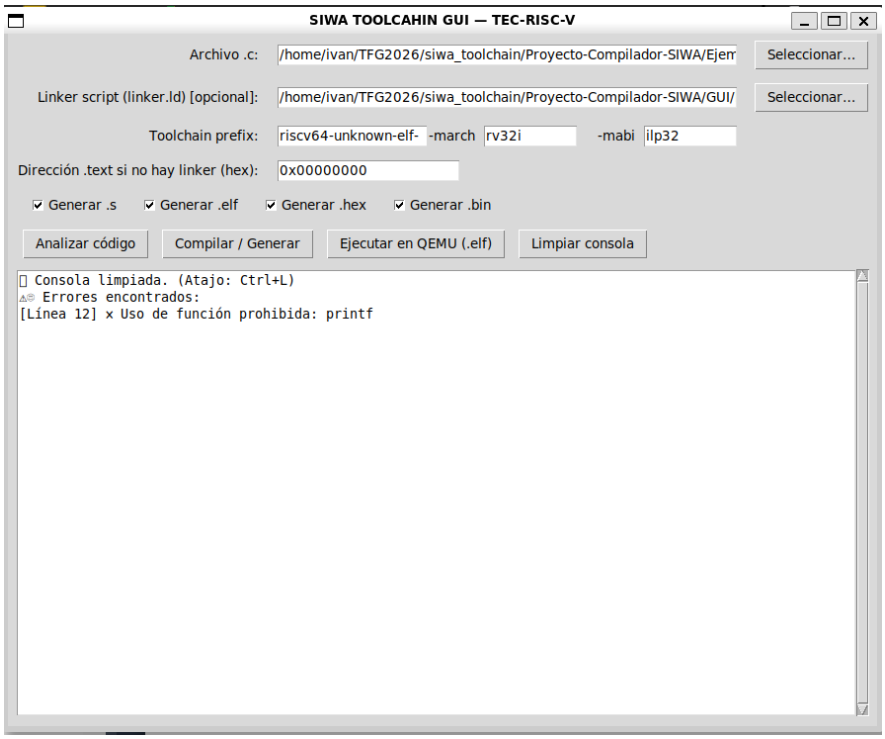

Figura 8: Resultado del programa en C utilizado para pruebas del toolchain en el GUI.

En el ejemplo mostrado en la figura 8, el analizador detecta el uso de la función `printf` en la línea 12 del programa. La herramienta reporta este problema en la consola mediante un mensaje de error que indica tanto la línea del código donde ocurre la instrucción como el tipo de restricción que se está violando.

Este mecanismo permite al usuario identificar rápidamente el uso de funciones incompatibles con el entorno embebido del sistema SIWA, facilitando la corrección del programa antes de iniciar el proceso de compilación.

Pruebas con programa compatible con arquitectura SIWA

Listing 3: Programa compatible con la arquitectura SIWA utilizando acceso a GPIO

```
#define GPIO_BASE 0x80000000
#define GPIO (*(volatile unsigned int*)GPIO_BASE)

volatile int a = 12;
volatile int b = 5;
```

```c
volatile int resultado;

void main() {

    // Realiza la suma
    resultado = a + b;

    // Muestra el resultado en el GPIO (por ejemplo, LEDs)
    GPIO = resultado;

    // Bucle infinito para mantener el resultado visible
    while (1);
}
```

Resultado con el programa compatible

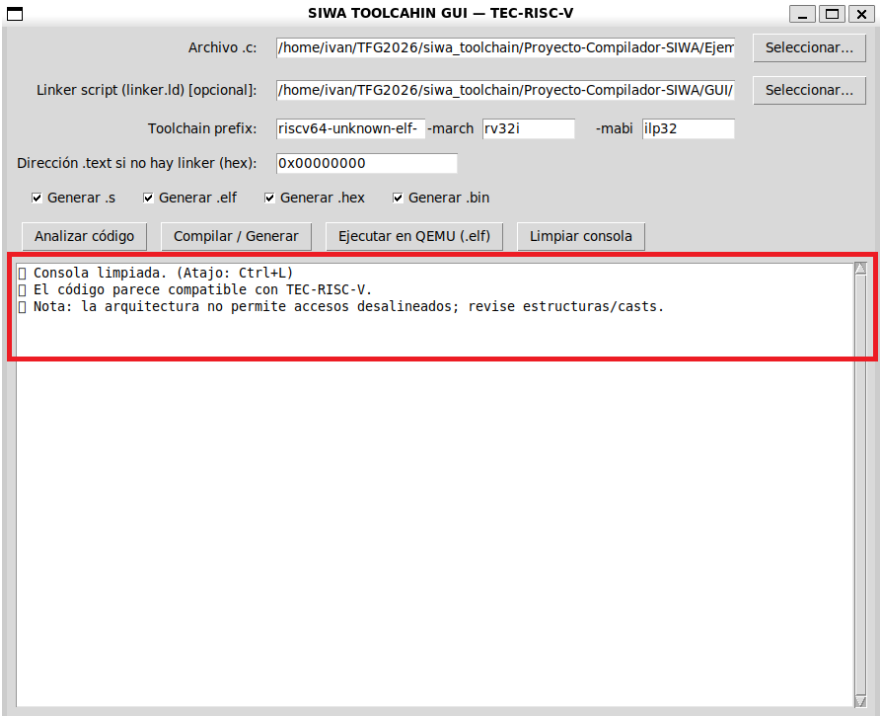

Figura 9: Resultado del programa en C compatible con arquitectura SIWA utilizado para pruebas del toolchain en el GUI.

En el ejemplo mostrado en la figura 9, el sistema indica que el código analizado parece compatible con TEC-RISC-V. Esto significa que el programa no presenta violaciones a

las reglas definidas por el entorno del microcontrolador y puede continuar con la siguiente etapa del flujo de trabajo, que corresponde al proceso de compilación y generación de los archivos binarios.

Adicionalmente, la herramienta muestra una nota informativa indicando que la arquitectura no permite accesos desalineados a memoria. Este mensaje funciona como advertencia para el usuario, recordando que las estructuras de datos y conversiones de tipo deben mantener una correcta alineación en memoria para evitar comportamientos inesperados durante la ejecución del programa.

- Botón de compilación: ejecuta el flujo de generación de archivos.

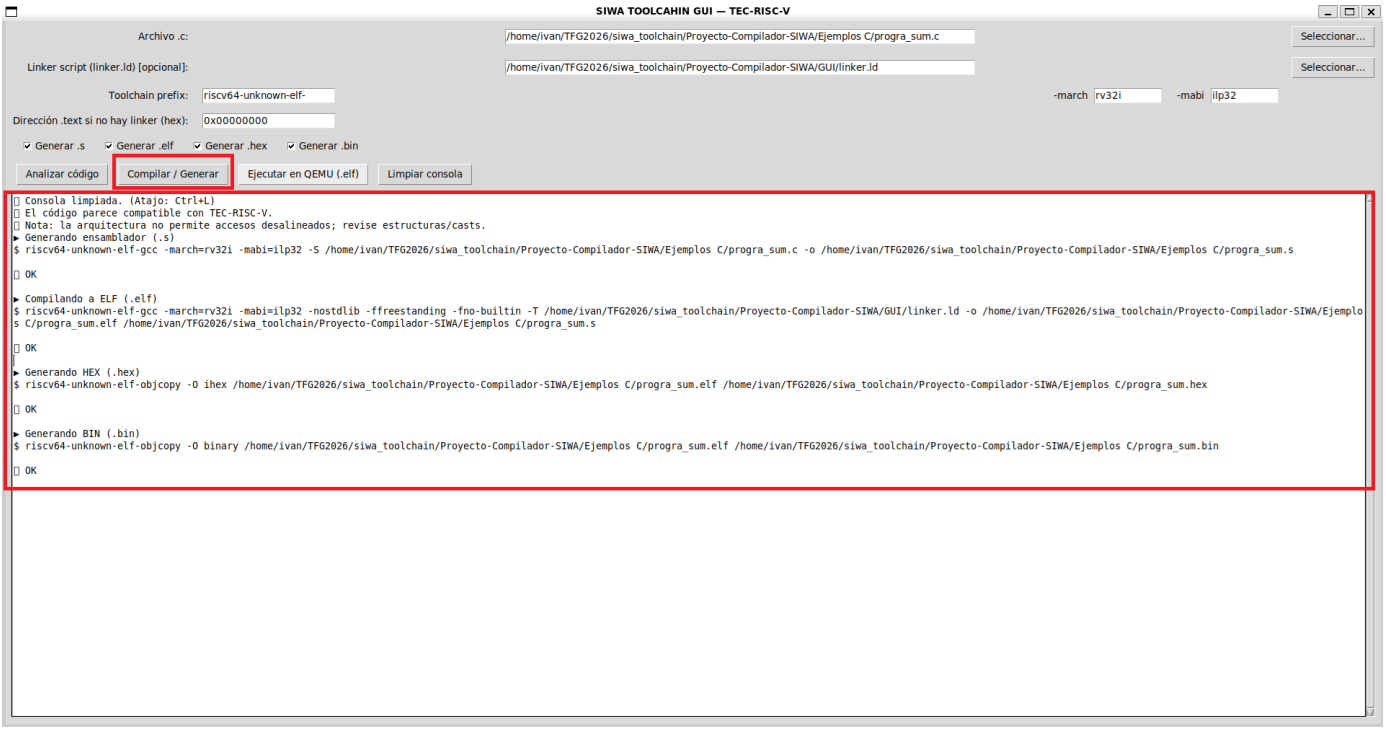

Figura 10: Resultado muestra los archivos creados y compilados por el GUI.

Durante esta etapa, el sistema ejecuta automáticamente las diferentes fases del toolchain de RISC-V para transformar el programa escrito en lenguaje C en los archivos necesarios para su ejecución en el microcontrolador. El progreso de este proceso se muestra en la consola de la interfaz.

Primero, el compilador genera el archivo de ensamblador (`.s`) a partir del archivo fuente en C utilizando el compilador `riscv64-unknown-elf-gcc` con los parámetros correspondientes a la arquitectura RV32I y el ABI `ilp32`.

Posteriormente, el sistema compila el programa y genera el archivo ejecutable en formato ELF (`.elf`), utilizando el script de linker especificado por el usuario. Durante esta etapa se aplican opciones como `-nostdlib`, `-ffreestanding` y `-fno-builtin`, las cuales permiten compilar el programa en un entorno embebido sin sistema operativo.

Una vez generado el archivo ELF, la herramienta utiliza la utilidad `objcopy` para convertir el ejecutable a otros formatos necesarios para diferentes etapas del flujo de trabajo. En particular, se generan los archivos:

- `.hex`, utilizado para cargar el programa en memoria del sistema.
- `.bin`, que contiene la representación binaria directa del programa.

Cada una de estas etapas se muestra en la consola de la herramienta junto con el comando ejecutado y un mensaje de confirmación cuando el proceso finaliza correctamente. De esta forma, el usuario puede verificar que el programa ha sido compilado y convertido correctamente a los formatos necesarios para su uso en la arquitectura SIWA.

- Consola integrada: muestra los resultados, advertencias y errores del proceso.

Resultados de la compilación

Análisis del archivo .elf

Listing 4: Desensamblado del programa progra_sum.elf

```
progra_sum.elf:     file format elf32-littleriscv

Disassembly of section .text:

00000000 <main>:
   0:   ff010113                addi    sp,sp,-16
   4:   00812623                sw      s0,12(sp)
   8:   01010413                addi    s0,sp,16
   c:   000017b7                lui     a5,0x1
  10:   0007a703                lw      a4,0(a5) # 1000 <a>
  14:   000017b7                lui     a5,0x1
  18:   0047a783                lw      a5,4(a5) # 1004 <b>
  1c:   00f70733                add     a4,a4,a5
  20:   000017b7                lui     a5,0x1
```

```
24:   00e7a423                sw      a4,8(a5) # 1008 <
   resultado>
28:   000017b7                lui     a5,0x1
2c:   0087a703                lw      a4,8(a5) # 1008 <
   resultado>
30:   800007b7                lui     a5,0x80000
34:   00e7a023                sw      a4,0(a5) # 80000000
38:   0000006f                j       38 <main+0x38>
```

El código ensamblador mostrado corresponde al programa `progra_sum.elf` generado por el toolchain de RISC-V a partir del programa escrito en lenguaje C. El análisis del desensamblado permite verificar que el programa es compatible con la arquitectura del sistema SIWA.

En primer lugar, todas las instrucciones utilizadas pertenecen al conjunto base de instrucciones RV32I, el cual es el conjunto soportado por el microcontrolador SIWA. Entre las instrucciones utilizadas se encuentran:

- `addi` para operaciones aritméticas inmediatas.
- `lw` y `sw` para acceso a memoria.
- `add` para la operación de suma entre registros.
- `lui` para cargar direcciones en registros.
- `j` para realizar saltos incondicionales.

Estas instrucciones forman parte del conjunto base RV32I, por lo que pueden ser ejecutadas directamente por el procesador implementado en el sistema SIWA.

Además, el programa realiza accesos a memoria alineados mediante las instrucciones `lw` y `sw`. Las direcciones utilizadas para las variables (`0x1000`, `0x1004` y `0x1008`) corresponden a direcciones alineadas a palabra, lo cual es requerido por la arquitectura y evita accesos desalineados.

El programa también escribe el resultado en la dirección `0x80000000`, la cual corresponde a un periférico mapeado en memoria (GPIO). Este tipo de acceso es común en sistemas embebidos y es compatible con el modelo de memoria utilizado por el sistema SIWA.

Finalmente, el programa termina con un bucle infinito implementado con la instrucción de salto `j`, lo cual es adecuado en sistemas embebidos sin sistema operativo, donde el programa no debe retornar desde la función principal.

Por estas razones, se puede concluir que el código generado es compatible con la arquitectura SIWA y puede ejecutarse correctamente en el microcontrolador implementado en el sistema TEC-RISC-V.

Análisis del archivo .bin

```
00000000  13 01 01 ff 23 26 81 00  13 04 01 01 b7 17 00 00  |....#&..........|
00000010  03 a7 07 00 b7 17 00 00  83 a7 47 00 33 07 f7 00  |..........G.3...|
00000020  b7 17 00 00 23 a4 e7 00  b7 17 00 00 03 a7 87 00  |....#...........|
00000030  b7 07 00 80 23 a0 e7 00  6f 00 00 00 00 00 00 00  |....#...o.......|
00000040  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00001000  0c 00 00 00 05 00 00 00                           |........|
00001008
```

Figura 11: Resultado del .bin compilado por el toolchain.

El archivo `.bin` generado por el toolchain corresponde a la representación binaria directa del programa compilado para la arquitectura RISC-V utilizada por el sistema SIWA. A diferencia del formato ELF, el archivo binario no contiene encabezados ni información de metadatos; únicamente almacena la secuencia de bytes que deben cargarse en memoria para que el procesador pueda ejecutar el programa.

Al analizar el contenido hexadecimal del archivo binario se observa que los primeros bloques de memoria corresponden a las instrucciones del programa generadas a partir del código fuente en C. Cada grupo de cuatro bytes representa una instrucción de 32 bits del conjunto de instrucciones RV32I, el cual es el conjunto base soportado por el microcontrolador implementado en el sistema SIWA.

Las instrucciones presentes en el archivo corresponden a operaciones básicas como carga y almacenamiento de datos (`lw`, `sw`), operaciones aritméticas (`add`, `addi`), carga de direcciones (`lui`) y saltos (`j`). Todas estas instrucciones pertenecen al conjunto base RV32I, por lo que son completamente compatibles con la arquitectura del sistema.

Además, los accesos a memoria observados en el código utilizan direcciones alineadas a palabra, lo cual cumple con las restricciones de acceso de memoria del procesador SIWA, que no permite accesos desalineados.

En las direcciones de memoria cercanas a `0x1000` se encuentran los valores iniciales de las variables globales utilizadas por el programa, lo cual confirma que el archivo binario incluye tanto las instrucciones ejecutables como los datos necesarios para la ejecución del programa.

Por estas razones, el archivo `.bin` generado es compatible con la arquitectura SIWA y puede ser cargado directamente en la memoria del sistema o utilizado para inicializar la memoria de un diseño implementado en FPGA.

Procedimiento general de uso

El uso de la herramienta puede resumirse en los siguientes pasos:

1. Abrir la aplicación SIWA TOOLCAHIN GUI.
2. Seleccionar el archivo fuente en lenguaje C.
3. Seleccionar opcionalmente un archivo `linker.ld`.
4. Verificar o ajustar el prefijo del toolchain.
5. Confirmar los parámetros `-march=rv32i` y `-mabi=ilp32`.
6. Elegir los archivos de salida que se desean generar.
7. Ejecutar primero el análisis del código.
8. Si no hay errores críticos, ejecutar la compilación.
9. Revisar la consola para verificar el resultado del proceso.
10. Si se generó correctamente el archivo `.elf`, ejecutar opcionalmente la simulación en QEMU.

Uso del análisis de código

La opción de análisis permite revisar el archivo fuente antes de iniciar la compilación. Esta función detecta restricciones asociadas a la arquitectura SIWA, tales como el uso de funciones no soportadas, direcciones inválidas, punteros sin `volatile` e instrucciones no compatibles con RV32I.

Se recomienda ejecutar siempre esta etapa antes de compilar, ya que permite corregir errores tempranamente y evitar la generación de archivos inválidos.

Generación de archivos de salida

Luego del análisis, la herramienta puede generar distintos tipos de archivos:

- `.s`: archivo en lenguaje ensamblador.
- `.elf`: archivo ejecutable enlazado.
- `.hex`: representación hexadecimal del ejecutable.
- `.bin`: binario plano para carga directa.

Cada uno de estos formatos cumple una función dentro del flujo de desarrollo del proyecto SIWA.

Ejecución en QEMU

Si el sistema tiene instalado `qemu-system-riscv32`, la herramienta permite ejecutar el archivo `.elf` en un entorno de simulación. Esta opción resulta útil para realizar pruebas preliminares del programa antes de cargarlo en hardware real.

Para utilizar esta función, primero debe haberse generado correctamente el archivo `.elf`.

Recomendaciones de uso

Para un uso adecuado de la herramienta se recomienda:

- Verificar que el toolchain de RISC-V esté correctamente instalado.
- Confirmar que el prefijo configurado coincida con las herramientas disponibles en el sistema.
- Utilizar un archivo `linker.ld` cuando se requiera una organización precisa de memoria.
- Revisar los mensajes de la consola después de cada operación.
- Ejecutar primero el análisis y luego la compilación.

Conclusión

La herramienta SIWA TOOLCAHIN GUI proporciona una interfaz práctica para automatizar el flujo de validación, compilación, conversión y simulación de programas destinados al microcontrolador SIWA. Su uso simplifica la interacción con el toolchain de RISC-V y facilita el desarrollo de software compatible con la arquitectura RV32I implementada en el proyecto.